Project Heartbeat's Embedded System Design

Jeffrey A. McCormick, *Student, Walla Walla University*

***Abstract*— This project seeks to use an analog to digital converter (ADC) to determine the heartrate of a person from an analog electrocardiogram (EKG) waveform.**

I. INTRODUCTION

SIGNAL processing is at the foundation of many breakthrough technologies. From car radios to medical scanning systems, most of all sensor equipment is used to measure continuous "real-world analog signals". [3] By using an ADC, those analog signals can then be processed digitally on, say, a computer or calculator or an FPGA.

For the embedded system called Project Heartbeat, an ADC will be used to convert an analog heartbeat signal and use the digital representation of it to compute and display for the user an extrapolated beats per minute (BPM) measure.

II. DESIGN METHODS: HARDWARE

The general approach to designing the hardware for Project Heartbeat relied heavily on the KL25Z FRDM board, ENGR 355 Dev board, and LCD specifications. Since their designs worked well together, making a custom Printed Circuit Board (PCB) that would seamlessly integrate into this system was paramount. A distinct benefit of using this approach was that I could program and test using the Dev board, while the parts and custom PCB were being delivered.

While the Dev board schematic was the pattern, there were several changes of note. First, the system had to be portable, meaning that the Project Heartbeat device would have to be battery powered. To accommodate this, a physical on-off switch and a voltage regulator were added to the schematic. The supplied voltage from the regulator was 3.3V to be used as power for the rest of the system.

Second, in order to program the MKL25Z128 microcontroller (MCU), a 10-pin header Serial Wire Debug (SWD) interface was added. The FRDM board would connect to this interface but only use six of the ten pins. Per the FRDM KL25Z Schematic indicating that the SWD clock and digital input/output pins should go on Port A pins 0 and 3, respectively, some of the switches were displaced to other pins.

Third, the five switches were still connected to Port A of the MCU so that they could have interrupt capabilities, but they were rearranged. The layout of the PCB was to have the five buttons arranged like a Swiss cross with SW1 centered and the others ordered sequentially clockwise around it. By this schematic arrangement, their traces on the PCB were more organized, taking up less space and reducing the risk of layout mistakes.

Fourth, six GPIO pins were placed on Port D, because that gives the designer maximum future project flexibility in case they wish to use interrupts with such pins.

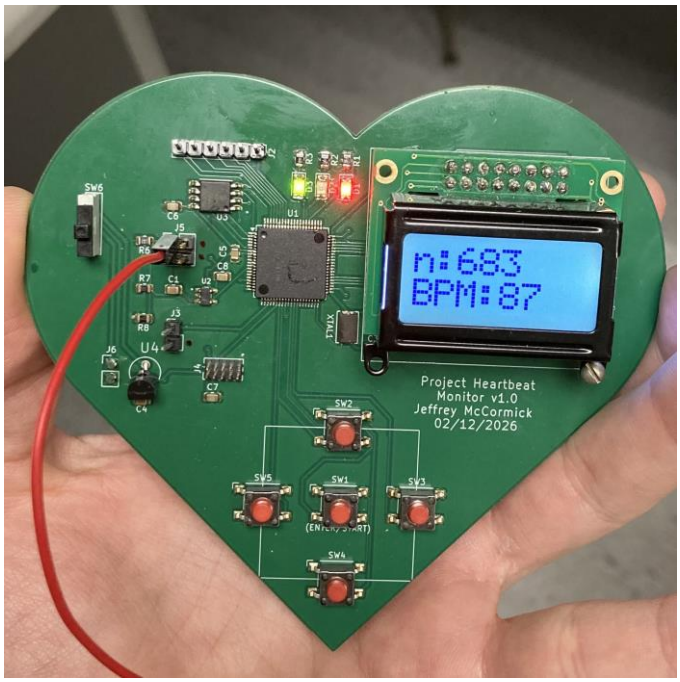
Fifth, an EEPROM memory chip was added. Again, this was chosen with future projects by the designer in mind, providing a way for users to read and write data for long-term storage without burdening the microcontroller's memory.

Sixth, the ADC input was designed for both DC and AC coupling. The AC coupling input passes through a voltage divider such that a signal of 0V AC enters the buffering source follower op amp, delivering an input of 1.65 V into the ADC. DC inputs are simply buffered without the scale shift.

As good practice, coupling capacitors were placed near the power pins of the MCU, SWD interface, LCD interface, ADC source follower op amp, and EEPROM. These were chosen to dampen any voltage fluctuations in the 3.3V power and to reduce the risk of noise adding to the signals.

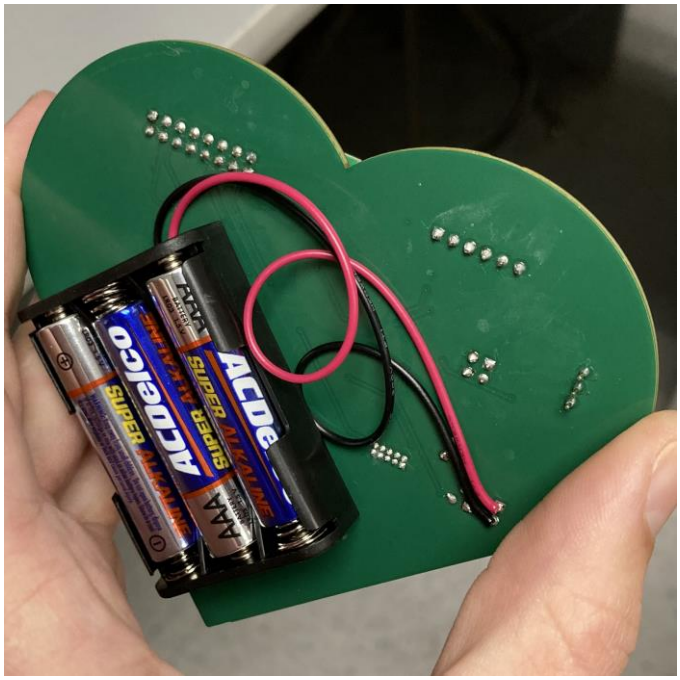
The PCB layout was designed for an easy and intuitive user experience as well as convenient debugging and flexible application. Many people in this modern digital era are familiar with Gameboy or other handheld Nintendo products, so the Project Heartbeat board layout was modeled after them. The hope is that a user can figure out how to use the board by drawing on this familiarity of an LCD screen with buttons below it. Since the primary use case of the board was to detect and calculate heartbeat behavior, it seemed fitting to let the user's intuition know that by making the board heart shaped. Although it may still be encased within a plastic covering, the board could still be used pleasantly without the covering by designers, developers, and debuggers. Therefore, the size of the board was made approximately 4 inches by 3 inches to fit comfortably in an average person's hand(s), even though a more compact layout could have had the same electronic functionality. With a less compact layout, the board's components can be easily accessed by a developer trying to debug the circuitry.

IMAGE 1
THIS IS THE FRONT VIEW



McCor1.jpeg

IMAGE 1
THIS IS THE BACK VIEW



McCor2.jpeg

IV. DESIGN METHODS: SOFTWARE

The software of Project Heartbeat was made for the primary use case of receiving a continuous analog heartbeat signal and output the BPM corresponding to the interval between the last two beats sensed. To make debugging easier, a modular approach was taken. The "main" function served as the testbench code for each function as they were being developed. This strategy ensured that any errors in the inputs and outputs of a function were caught early and corrected. Artificial intelligence (AI) was used as a tool to expedite each function's code generation by prompting with specific desired inputs, outputs, purpose, and role in the software and project as a whole. Because the software functions are not placed in order of their dependency in the "Lab_8_Code.c" c-language code file, the following explanations of the code will be presented in order of their dependency from input to output.

A. Enable the Clocks

This ENABLE_CLKS() function enables the default system clock signal in Clock Setup 1 (48MHz) to go to Ports A, B, C, and E of the MCU. It is mostly just to ensure the code is completely enabled or disabled by whether the function is called or not, respectively.

B. Initialize ADC

The Init_ADC() function sets the ADC to:

1) 12-bit conversion standard
2) Clock division by 4
3) Clock source of ADICLK (default)

It also enables the ADC0 conversion complete flag (COCO) to trigger an interrupt to be handled in the ADC IRQ Handler.

C. Initialize SysTick Interrupts

For use in other functions as an independent timer from the PIT, this Init_SysTick() function simply sets the ticks to occur once every 1ms and trigger an interrupt with it.

D. Initialize PIT and Define Its IRQ Handler

Similar to the SysTick initialization, the PIT_init() function serves to enable interrupts which occur exactly once per 1ms. It's PIT_IRQHandler() function can then increment the two counters used by the buttons for debouncing and the heartbeat interval calculator for measurement. A green LED is linked to every PIT flag raising so that the user can know if the device is on and running.

E. Initialize Buttons and Define Their Port A ISR Handler

Init_Buttons() sets all the buttons to input GPIOs with pull-up resistor setting configured in the MCU and with interrupt capabilities. Then, because the buttons are on Port A, the PORTA_IRQHandler() function triggers when a button is pressed and uses the system millisecond counts from the PIT to debounce the button press and set a program flag as true.

F. Initialize the DAC

This Init_DAC() function simply sets the DAC as active and referencing the same reference voltage as the other components.

G. Define ADC IRQ Handler

This ADC0_IRQHandler() function operates in two modes. Mode 1 is the most useful mode, because it sets the DAC to be an output for reconstruction of the input heartbeat signal. Mode 2 sets the DAC as a simulator essentially so it does not need to be tied to the ADC's output.

The function triggers whenever the input converted signal crosses the threshold and hysteresis digital voltage values. They can be set to DC and AC coupling, which only changes whether the interrupt is triggered on the rising or falling edge of a beat. See code comments for examples.

A red LED is used as an indicator that a conversion happened.

H. Get the Heartbeat from the ADC Signal

When a conversion completes, this get_heartbeat() function simply captures the current PIT count in the interval between beats and resets the count to be ready for the next beat. The PIT increments every 1ms, so the PIT counts that happened in the interval is equivalent to the number of milliseconds between the beats.

I. Initialize LEDs

This Init_LEDs() function simply sets the LEDs to GPIO outputs so that they can be used as indicator lights.

J. Get BPM from the Heartbeat Intervals in Milliseconds

This get_bpm() function is the convert the beat interval in ms found by the get_heartbeat() function into the calculated BPM corresponding to a period equal to the length of that interval. While this lends itself to greater fluctuation in the calculated BPM for small deviations from an average input signal's BPM, it allows for processing as close to present as possible so that a "live" data stream can be viewed by the user. The calculation uses integer math to save computation time and memory space of the MCU. Again, the goal is as close to a real-time update as possible.

K. Initialize LCD and Display a String of Relevant Data

Using the unaltered "lcd_lib_4bit_20b.c" LCD library of commands file written by Dr. Larry Aamodt, the LCD_init() function initializes the LCD. Displaying a string on any selectable line of the LCD is the purpose of the displayString() function. By supplying a character array to display and the line selection of 0 for the top line and 1 for the bottom line, the LCD will display only the first 8 characters in the string on the selected line. This function allows explicit developer control over what is shown on the LCD screen.

L. The Main Function

The main function calls all of the above functions as needed for scope or simulator modes of the device. Within a while loop running faster than the 1ms PIT or SysTick interrupts, get_heartbeat() checks for a beat from the ADC, a simple LCD user interface tells users what the state of the program is, and running logic critical for "live" updates of the LCD ensures that recently captured and processed signal data is displayed on the LCD without race conditions. With the large amount of interrupts and the random nature of when a signal could enter the ADC, it is important that the transmission to the LCD finish before seeking another one. This could potentially cause under counting, but not if the signal period is greater than 2ms, which is true for heartbeat signals even up into the arithmetic heart rates (500+ BPM). Another timing aspect of the running logic is that the LCD should only update the display when the measured period between the beats actually changes. This task reduction results in more stable display behavior.

V. RESULTS

A demonstration of the Project Heartbeat board working as a live heartrate monitor in action was performed on March 17, 2026 on Jeffrey McCormick. Medical-grade EKG pads were placed at each shoulder and his left leg. The leads connected to an isolated amplifier circuit board made by Dr. Larry Aamodt, which then sent the signal ranging from 0v to 3.3V to the ADC input pins on the Project Heartbeat board.

The initial result was very high BPM measurements. To see what thresholds to adjust the code to, an oscilloscope was connected to the Project Heartbeat ADC pins. After increasing the threshold, the board read the live heartbeat signals within 5 percent error of the value from a standard Oura Ring 4 simultaneously measuring the live heart rate from McCormick's index finger.

VI. CONCLUSION

Overall, Project Heartbeat as a physical, custom embedded system met its essential requirements as specified by instructor Dr. Natalie Smith-Gray. It is "a hand-held portable instrument that can analyze a periodic analog input signal (voltage)[1] While its embedded system demonstrates promise, it can still be improved by adding a filtering or stream-averaging scheme to the software, more functions like HRV measurements, and a visually and physically appealing plastic case to hold it.

REFERENCES

[2] N. Smith-Gray, "ENGR 355 Course Project," unpublished. Winter 2026. https://class.wallawalla.edu/d2l/le/lessons/379831/topics/4176668

[3] Nintendo Co. *Game Boy*. https://nintendo.fandom.com/wiki/Game_Boy (Accessed 18 Mar 2026)

[4] IC Online, "ADC vs DAC Key Differences and Modern Electronics Applications," https://ic-online.com/blog/post/adc-vs-dac-key-differences-and-modern-electronics-applications (Accessed 18 Mar 2026)